HW uart_loopback

0. summary

over sampling 16 / 동작 주파수 / margin 43.75% / 어떤 테스트 케이스에 대해서 PASS

1. 목 적

2. UART Reference
- UART 통신 프로토콜(통신 방향 half duflex 인지 뭔지..)
  -> 해당 설계의 데이터 통신 방식 ONE BIT mode
- Margin 개념 설명
- PC에서 들어오는 rx 신호를 9600bps *16 baud rate로 샘플링
  ➔ Sampling rate를 더 높일 수 있는데 왜 16 baude rate가 최선인지

3. 블록 다이어그램

4. ASM 차트

5. 시뮬레이션

6. 트러블 슈팅

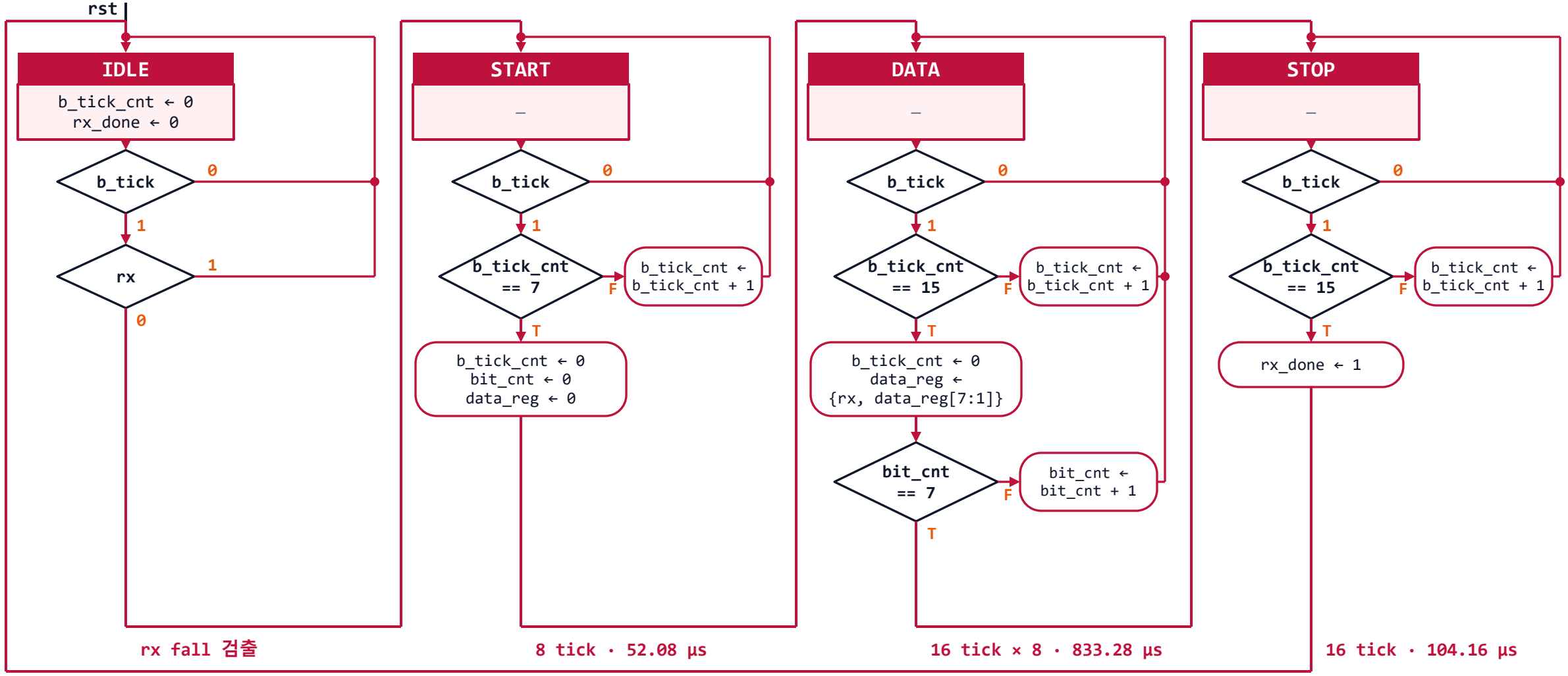
7. 시연

7. 결론 및 고찰
결론 – UART LOOP BACK 구조로 UART TX / RX 모듈 검증 완료
  - rx glitch 상황에서 모듈 구조 개선으로 데이터 오입력 방지 개선
고찰 – ONE BIT MODE로 인해서 Data sampling 구간에서 글리치 발생 시, 데이터 값이 일치 하지 못함
  - 향 후 3개의 sampling을 통해 데이터 글리치에 올바른 데이터 값이 전송 될 수 있도록 만들면 좋을 것 같음
  - 3tick 이상의 클리치에 관해선 data error 같은 정정코드를 보내서 device에서 다시 보낼 수 있도록 하는 모듈 구조 개선이 필요할 것
으로 보임 (에러 정정 코드 같은 것)

8. APPENDICX
- 글리치 장표, 데이터 글리치

uart_rx ASM: Align to Start Center, Then Sample Every 16 Ticks

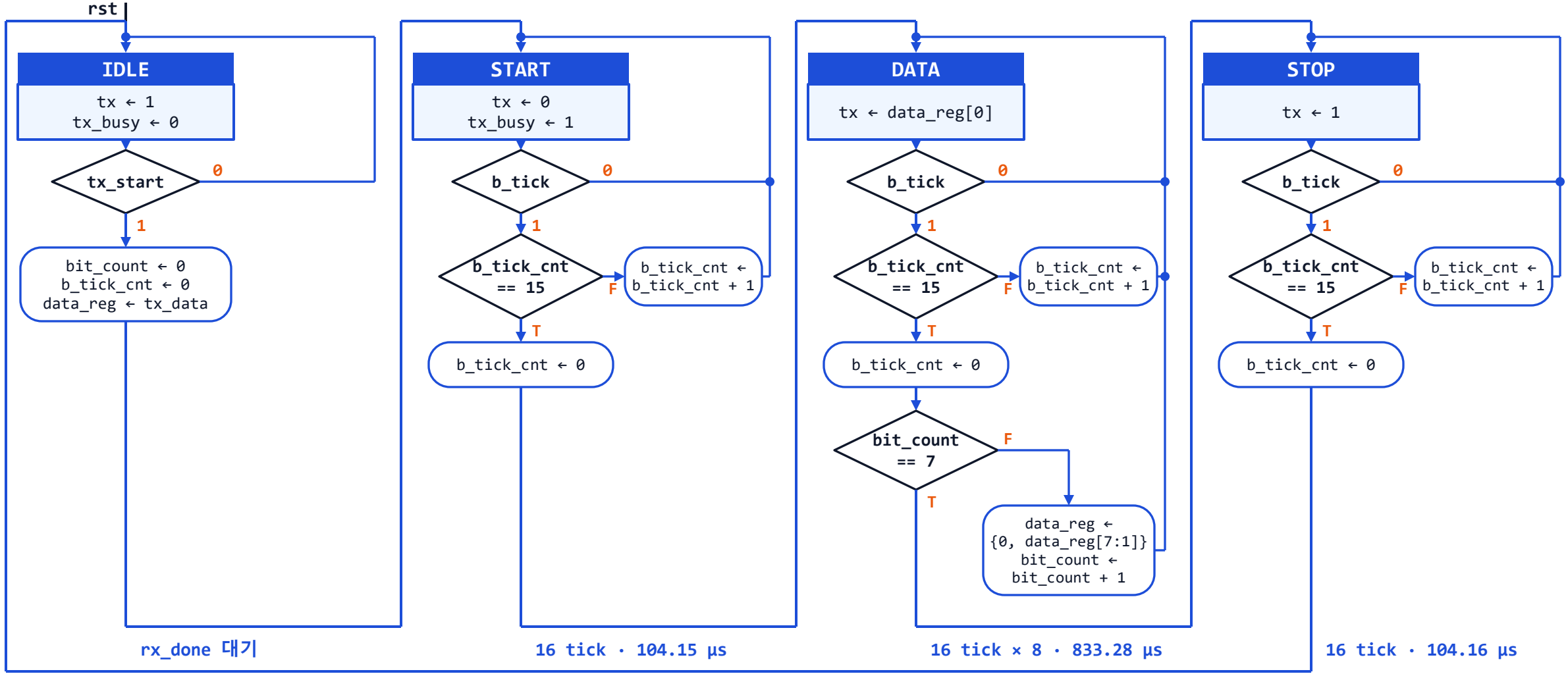
03



State box (무조건 동작) · Decision box · Conditional output box · ● path merge · ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

uart_tx ASM: Hold Each Bit for 16 Ticks, LSB First

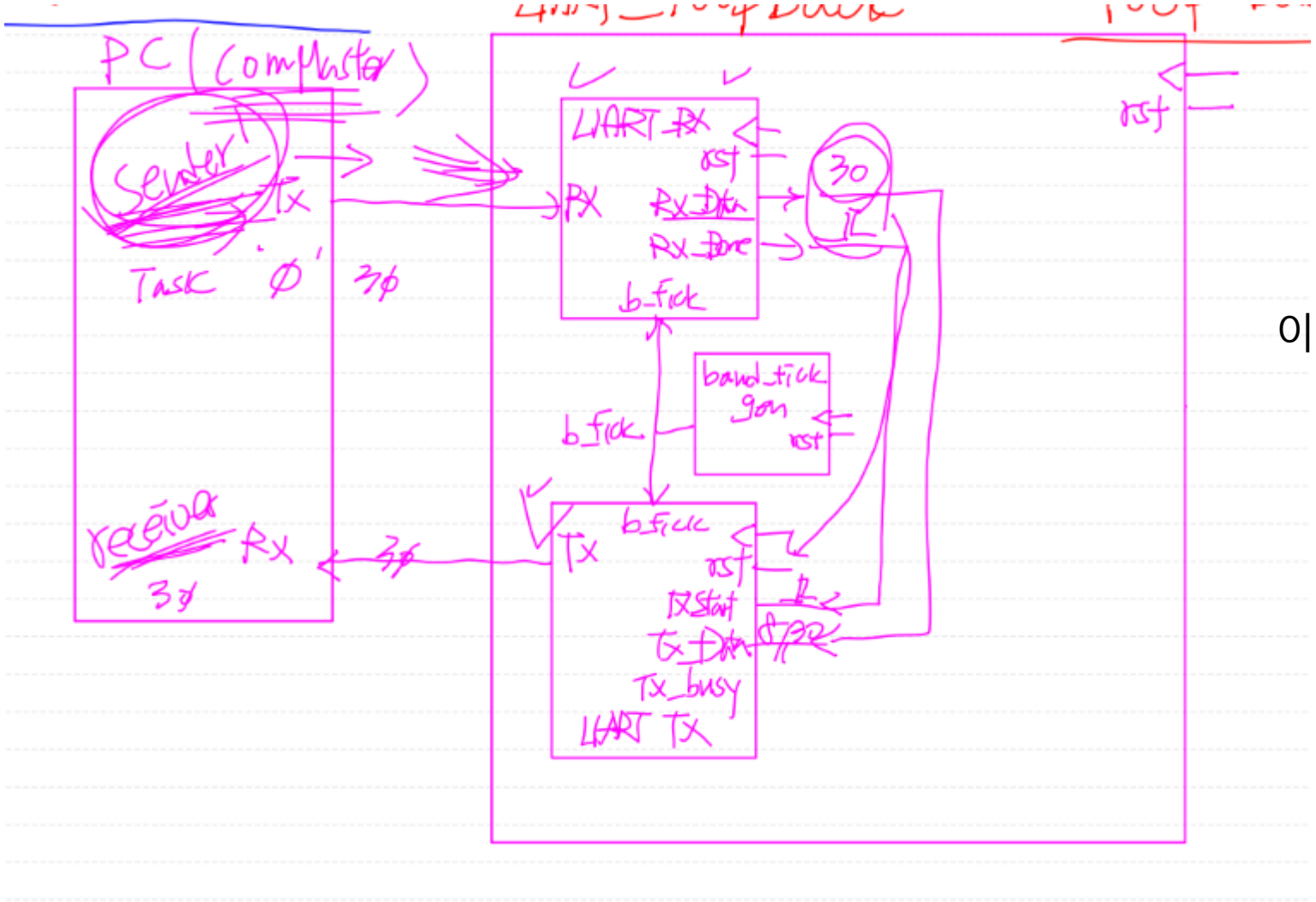
03



State box (무조건 동작) · Decision box · Conditional output box · path merge · ← 다음 clock edge 갱신 · T / F 조건 참 / 거짓

uart_loopback: RX Output Feeds TX Directly

01



이 구조로 블록다이어그램 만들어줄 것

UART_LOOPBACK 시나리오

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과
1	rx 프로토콜 동작 타이밍 분석	rx 1->0 9600bps 간격으로 데이터 송신(sendor) stop 0->1	rx_state, b_tick_cnt,스테이트 간 타이밍,	rx 1->0 과정에서 다음 b_tick 에서 start 로 변경 b_tick_cnt ==7에서 Data state 변화
2	Data sampling	8'h00~8'hFF 256 case input (rx_sender)	rx, rx_data_reg, tx, tx_data_reg	'DATA' state 구간에서 b_tick_cnt == 15 간격으로 rx_data가 shift 되면서 첫번째 LSB값 출력
3				
4				
5				

이런식으로 표를 채워 줄것.

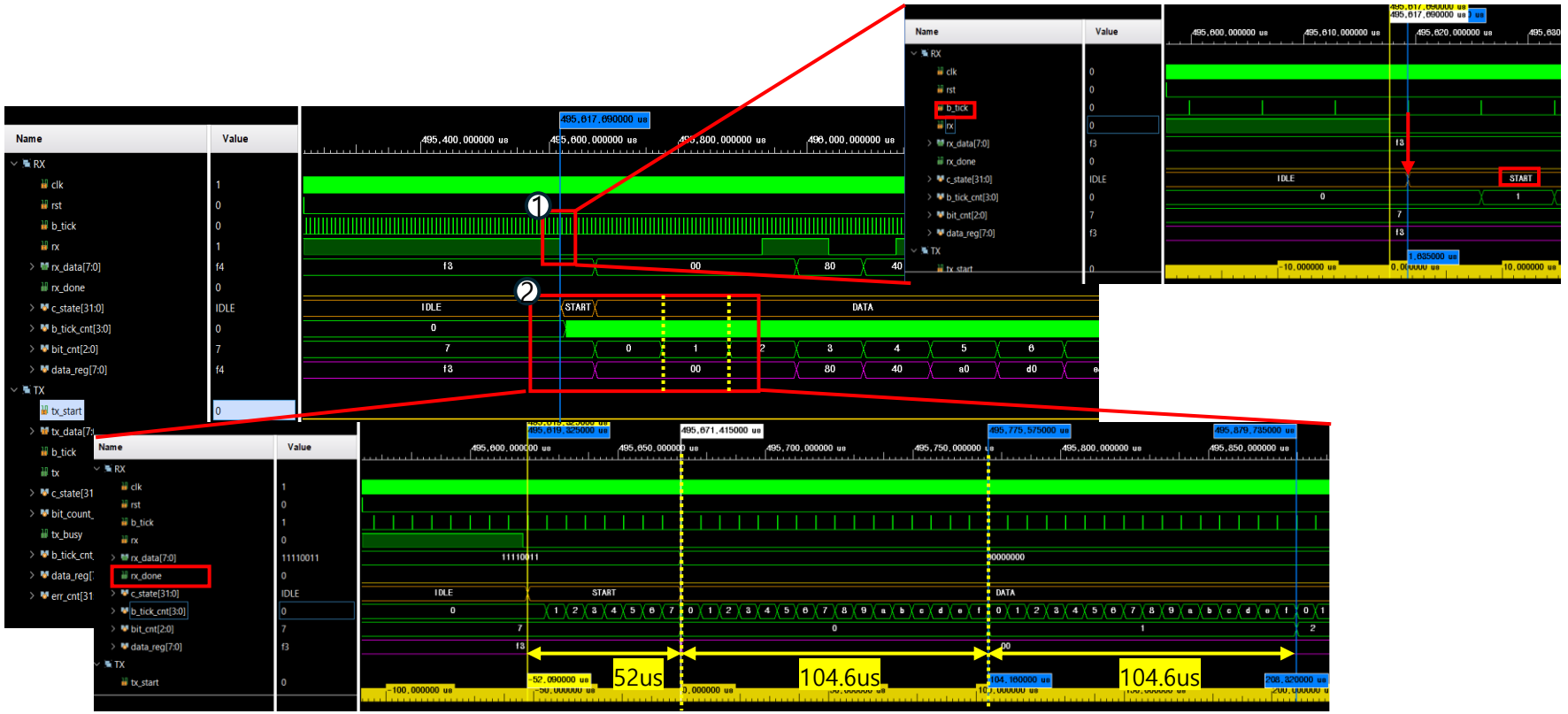
UART RX 프로토콜 동작 확인

확인 부분 – rx deassert 이후 uart_rx (IDLE → Start → DATA 구간)

결론

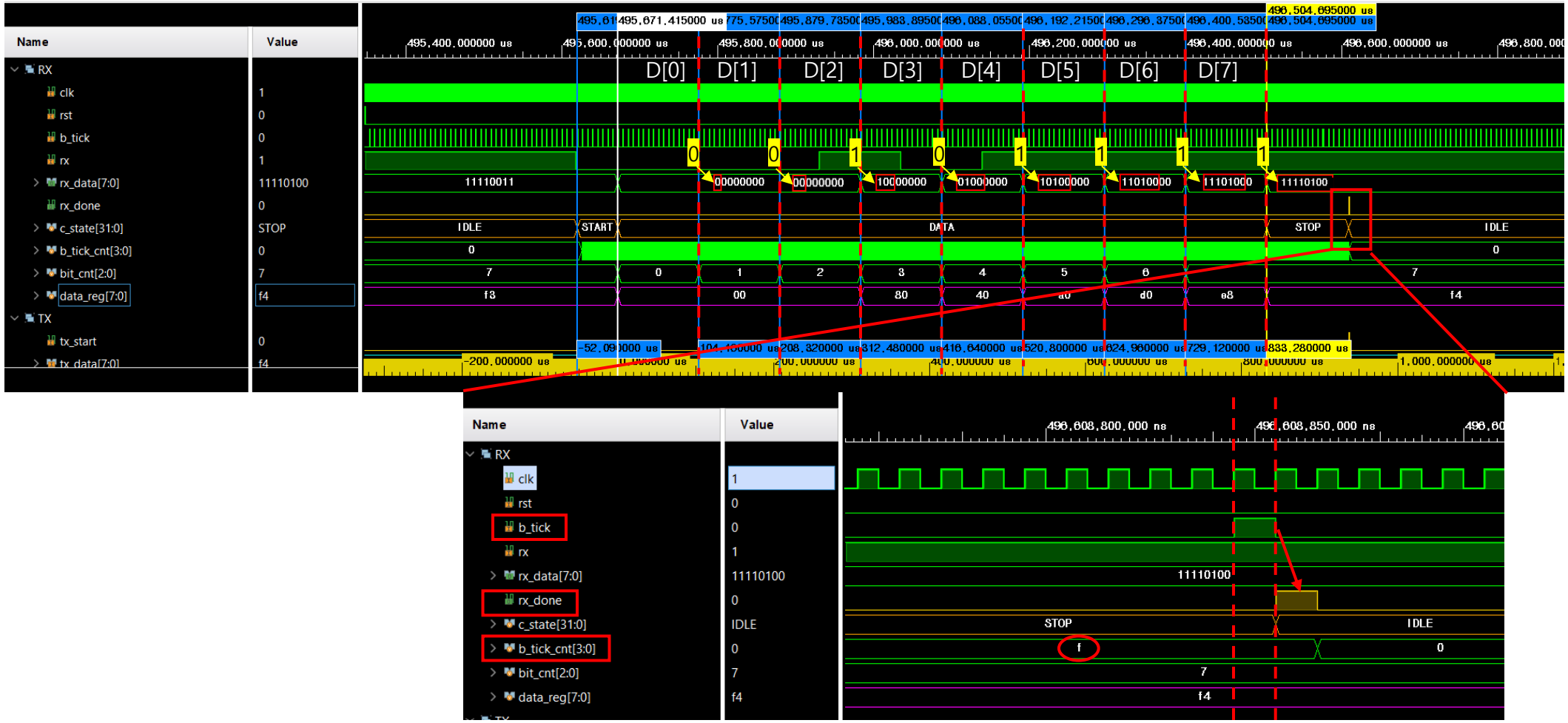
❶ rx deassert → b_tick이 1일 때, IDLE → Start

❷ Sampling timing 일치 (Start – 52us / Data – 104.6us)



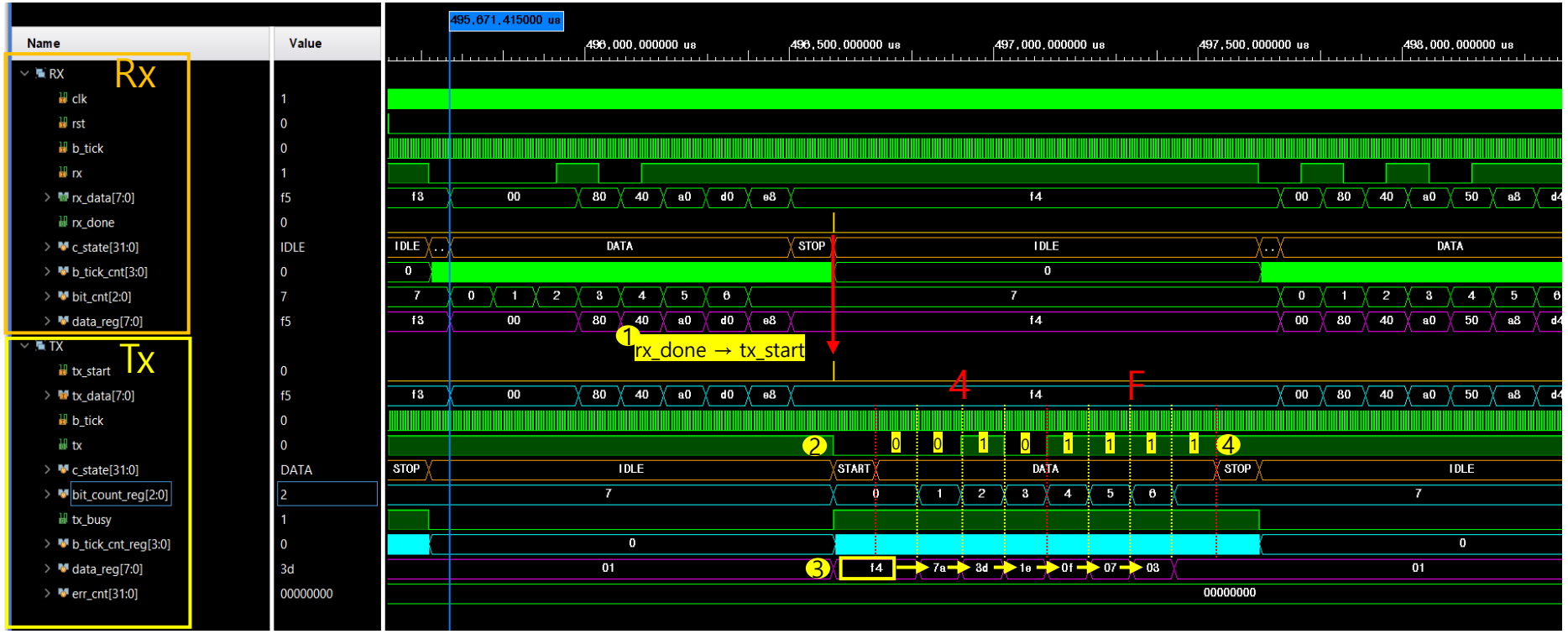
UART RX/TX 프로토콜 동작 확인
확인 부분 – Data Sampling 및 출력

결론 b_tick_cnt == 15 구간에서 Data shift 및 Stop 구간에서 8bit Data 수신
Stop state asm chart 만족



UART RX/TX 프로토콜 동작 확인
확인 부분 – 전체 TX 동작 FLOW

① Rx_done -> tx_start
② tx_start 이후, tx signal deassert
③ Start에서 rx_data update, 이 후 shift 되면서 tx line으로 데이터 전송
④ STOP state에서 tx signal assert



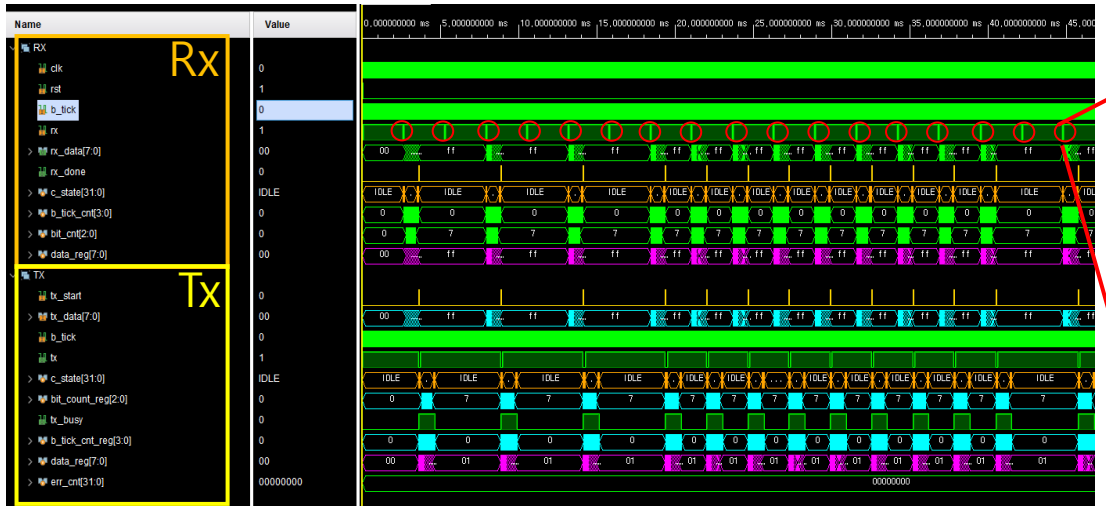

```
INPUT for 256 case Result
        256 case  pass :        256  fail :          0
```

256 input 결과 모두 pass

6. 트러블 슈팅 1

문제

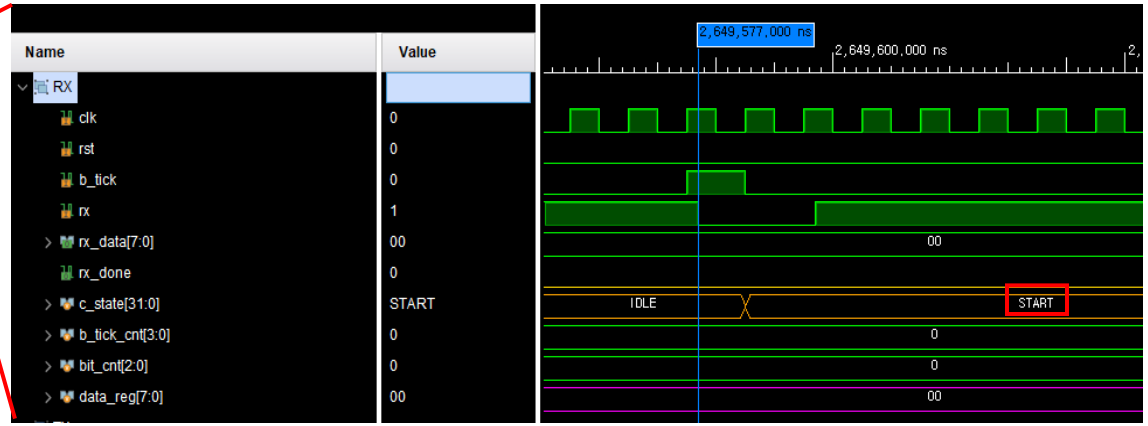
rx glitch 발생 시, data가 전송되는 문제
rx glitch test에서 Fail 검출




```
---- glitch burst / chattering (expect: ignored) ----
44774288 ns : PASS glitch w=200 ns x5 gap=1000 off-tick : ignored
47394798 ns : FAIL glitch w=1000 ns x8 gap=1000 off-tick : rx_done 1 (last ff), tx echo 1 (last ff)
---- glitch then real byte (expect: byte received once) ----
51149288 ns : PASS glitch w=200 ns off-tick + 104160 ns + data 55 : received 55
54938007 ns : FAIL glitch w=200 ns on-tick  + 104160 ns + data 55 : rx_done 2 (last ff), tx echo 1 (last aa)
58726827 ns : FAIL glitch w=200 ns on-tick  + 104160 ns + data d5 : rx_done 1 (last aa), tx echo 1 (last aa)
62723967 ns : FAIL glitch w=200 ns on-tick  + 312480 ns + data 55 : rx_done 2 (last fe), tx echo 1 (last ab)
67658547 ns : FAIL glitch w=200 ns on-tick  + 1249920 ns + data 55 : rx_done 2 (last 55), tx echo 2 (last 55)
GLITCH Result  pass : 6  fail : 17
```

원인 분석

uart rx state에서 rx falling 감지 순간 start 스테이트로 넘어감 bit_cnt 이후 바로 다음 state로 넘어감 -> 글리치에 대한 대응 코드가 존재하지 않음



6. 트러블 슈팅1

해결
start state에서 rx 0인 상태에서만
b_tick count를 counting 하도록 설계 변경

```
START: begin
    if (b_tick) begin
        if (b_tick_cnt == 7) begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = DATA;
        end else begin
            b_tick_cnt_next = b_tick_cnt + 1;
        end
    end
end
```

→

```
START: begin
    if(!rx)begin
      if (b_tick) begin

        if (b_tick_cnt == 7 ) begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = DATA;
        end

        else begin
            b_tick_cnt_next = b_tick_cnt + 1;
        end

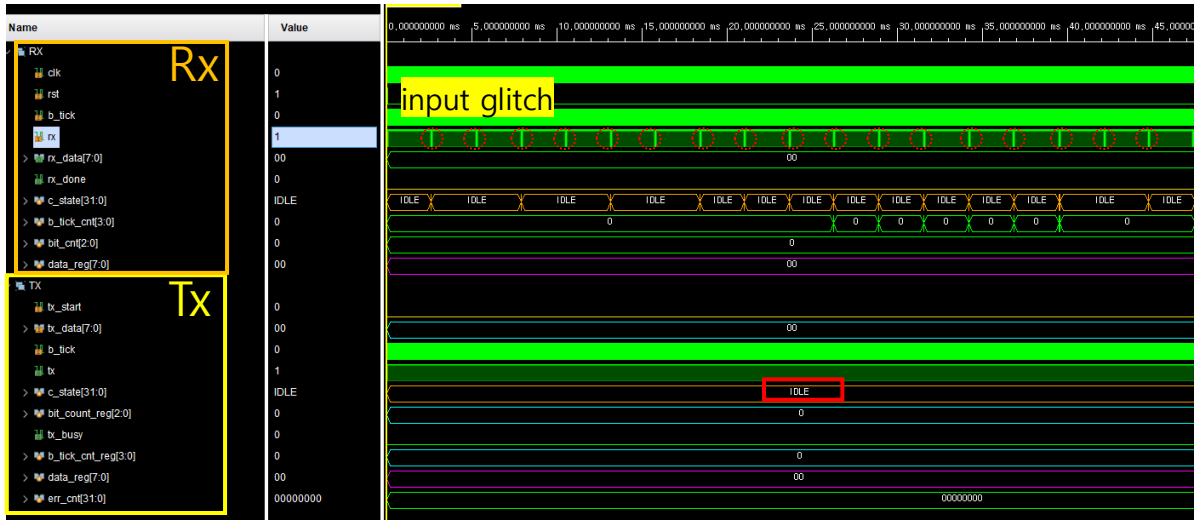
      end
    end
    else begin
            b_tick_cnt_next = 0;
            bit_cnt_next    = 0;
            data_next       = 8'h00;
            n_state         = IDLE;

    end

end
```

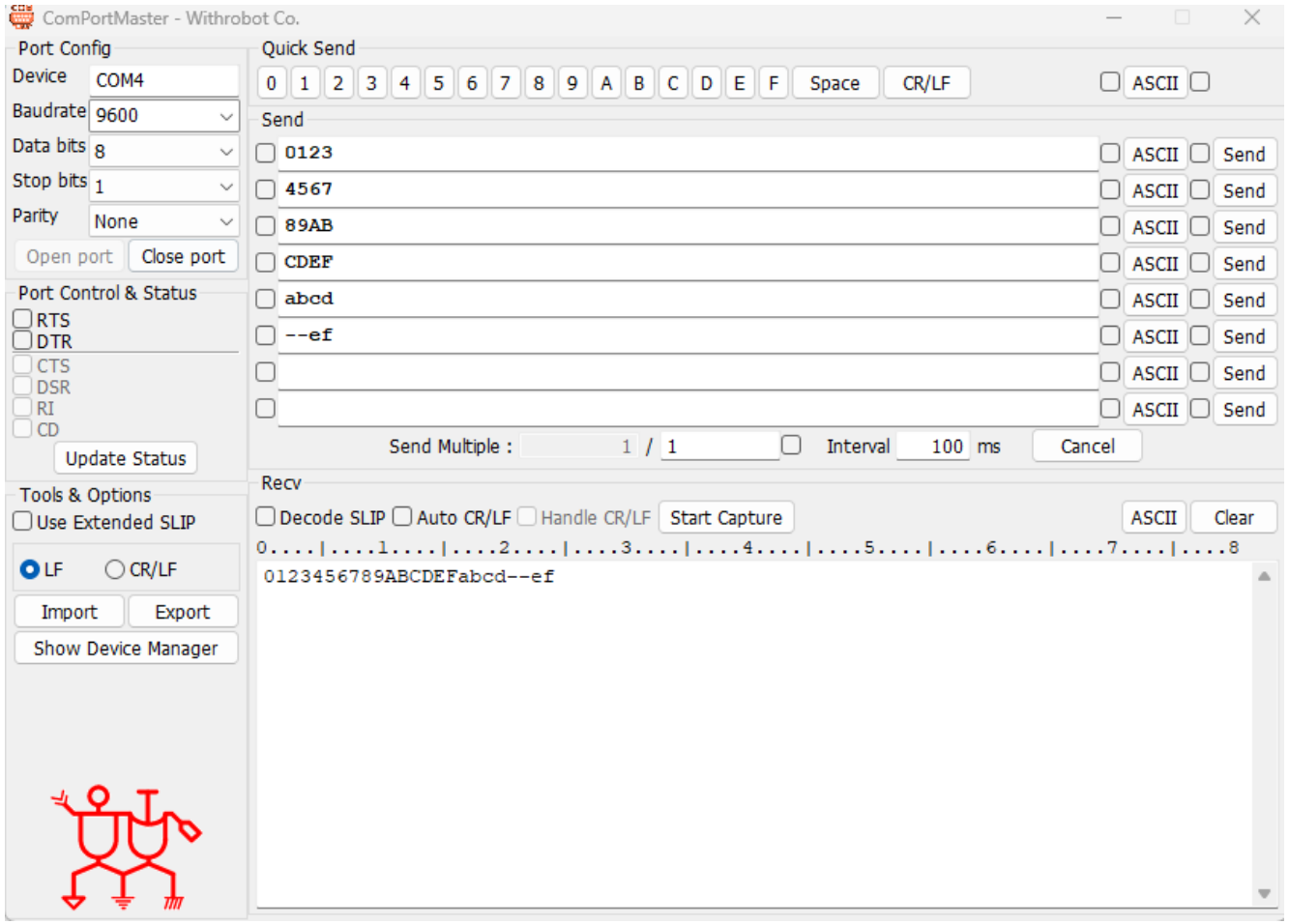
b_tick_count <7 상황에서 rx가 1이면 프로토콜 오동작이므로 다시 IDLE 상태로 전환

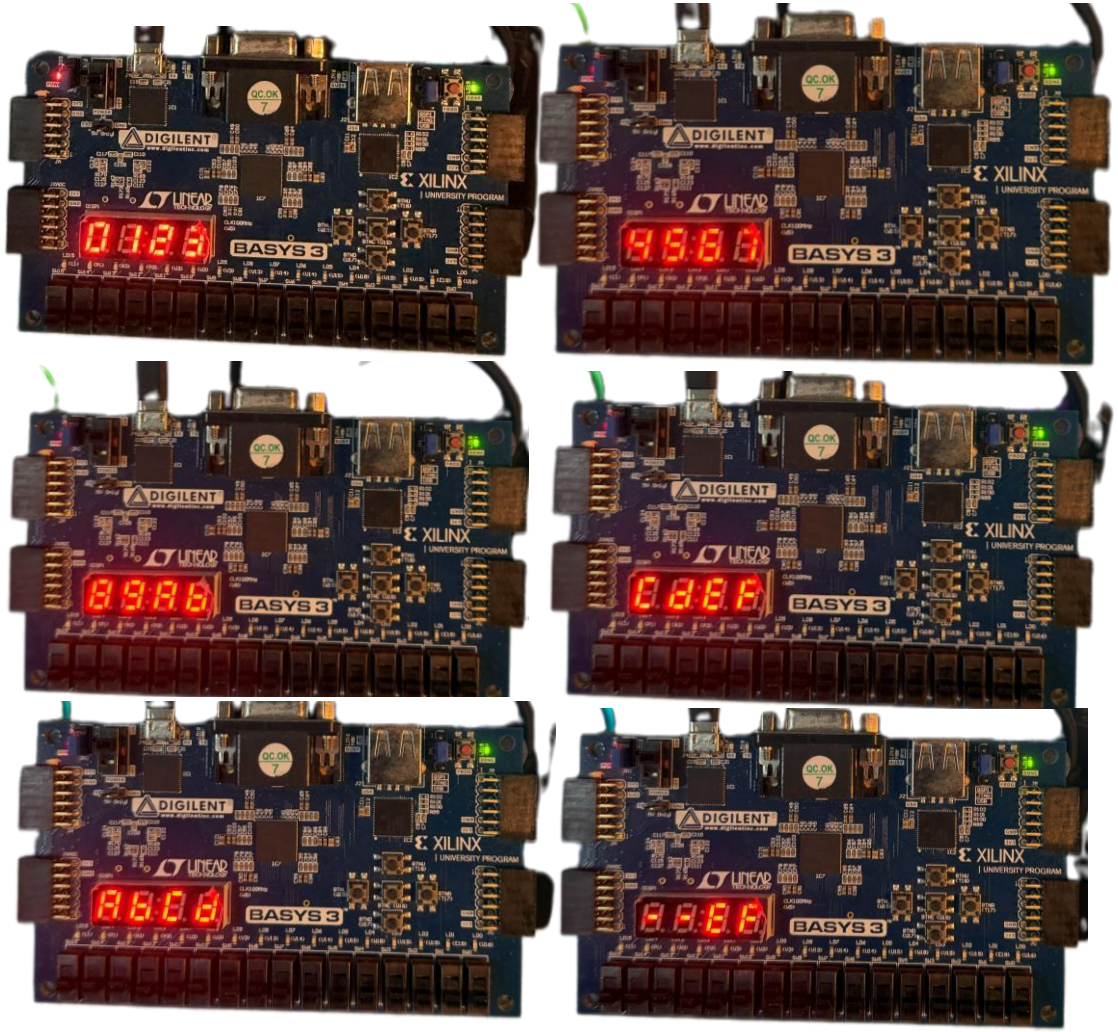
적용 결과
glitch 경우 PASS, 글리치 입력에도 Tx data 변화 x




```
39500618 ns : PASS glitch w=50000 ns x1 gap=0 off-tick : ignored
42164247 ns : PASS glitch w=50000 ns x1 gap=0 on-tick  : ignored
---- glitch burst / chattering (expect: ignored) ----
44774288 ns : PASS glitch w=200 ns x5 gap=1000 off-tick : ignored
47394798 ns : PASS glitch w=1000 ns x8 gap=1000 off-tick : ignored
---- glitch then real byte (expect: byte received once) ----
51149288 ns : PASS glitch w=200 ns off-tick + 104160 ns + data 55 : received 55
54938007 ns : PASS glitch w=200 ns on-tick  + 104160 ns + data 55 : received 55
58726827 ns : PASS glitch w=200 ns on-tick  + 104160 ns + data d5 : received d5
62723967 ns : PASS glitch w=200 ns on-tick  + 312480 ns + data 55 : received 55
67658547 ns : PASS glitch w=200 ns on-tick  + 1249920 ns + data 55 : received 55
GLITCH Result  pass : 23  fail : 0
```

시연 결과 – 한줄


ComPortMaster - Withrobot Co.
Port Config
Device COM4
Baudrate 9600
Data bits 8
Stop bits 1
Parity None
Open port
Close port
Port Control & Status
RTS
DTR
CTS
DSR
RI
CD
Update Status
Tools & Options
Use Extended SLIP
LF
CR/LF
Import
Export
Show Device Manager
Quick Send
0 1 2 3 4 5 6 7 8 9 A B C D E F Space CR/LF
ASCII
Send
0123
4567
89AB
CDEF
abcd
--ef
Send Multiple : 1 / 1
Interval 100 ms
Cancel
Recv
Decode SLIP
Auto CR/LF
Handle CR/LF
Start Capture
Clear
0123456789ABCDEFabcd--ef


DIGILENT
XILINX
BASYS 3